

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B.Harsha Vardhan	Reg. No.:	192425201
Section / Batch:	2024	Date:	10/07/26

Code of Conduct

I B.Harshavardhan(192425201)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __B.Harsha Vardhan

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

CODE:

```
import re

# ----- Regex patterns -----
NAME_LABEL = re.compile(r"Name\s*[:\-\s]*([A-Za-z]+(?:\s[A-Za-z]+){0,2})", re.I)
NAME_FALLBACK = re.compile(r"^[A-Z][a-z]+(?:\s[A-Z][a-z]+){1,2})\s*$", re.M)
EMAIL = re.compile(r"[\w.-]+@[ \w.-]+\.[\w.]+")
MOBILE = re.compile(r"(?:\+91[\-\s]?)?\b[6-9]\d{9}\b")
EXPERIENCE = re.compile(r"(\d+(?:\.\d+)?)\s*~+?\s*(?:years?)\s*yr(s)?", re.I)

SKILLS = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
SKILL_PATTERNS = {s: re.compile(r"\b" + re.escape(s) + r"\b", re.I) for s in SKILLS}

# ----- Extraction functions -----
def extract_name(text):
    m = NAME_LABEL.search(text) or NAME_FALLBACK.search(text)
    return m.group(1).strip() if m else "Not found"

def extract_emails(text):
    return EMAIL.findall(text)

def extract_mobiles(text):
    return MOBILE.findall(text)

def extract_skills(text):
    return [s for s, p in SKILL_PATTERNS.items() if p.search(text)]

def extract_experience(text):
    years = [float(y) for y in EXPERIENCE.findall(text)]
    return max(years) if years else 0.0

def parse_resume(text):
    return {
        "name": extract_name(text),
        "emails": extract_emails(text),
        "mobiles": extract_mobiles(text),
        "skills": extract_skills(text),
        "experience": extract_experience(text),
    }
```

```

def summary(c):
    return (
        f"Name      : {c['name']}\n"
        f"Email       : {' '.join(c['emails']) or 'N/A'}\n"
        f"Mobile      : {' '.join(c['mobiles']) or 'N/A'}\n"
        f"Skills      : {' '.join(c['skills']) or 'N/A'}\n"
        f"Experience  : {c['experience']:g} years"
    )

def is_eligible(c, min_exp=2):
    return c["experience"] >= min_exp and "Python" in c["skills"]

# ----- Sample resumes -----
resumes = [
    """Name: Ravi Kumar
    Email: ravi.kumar@gmail.com
    Mobile: +91 9876543210
    Skills: Python, SQL, Machine Learning
    Experience: 3 years in data analytics""",

    """Name: Anita Sharma
    Email: anita.sharma@yahoo.com
    Phone: 8123456789
    Skills: Java, SQL
    Total experience: 5 years as backend developer""",

    """Name: Suresh Reddy
    Email: suresh_r@outlook.com
    Mobile: 9988776655
    Skills: Python, NLP, Java
    Experience: 1 year internship experience, 1.5 yrs in NLP research""",

    """Name: Priya Nair
    Email: priya.nair@company.com
    Mobile: +91-7012345678
    Skills: Python, NLP, Machine Learning
    Experience: 4+ years in AI development""",
]

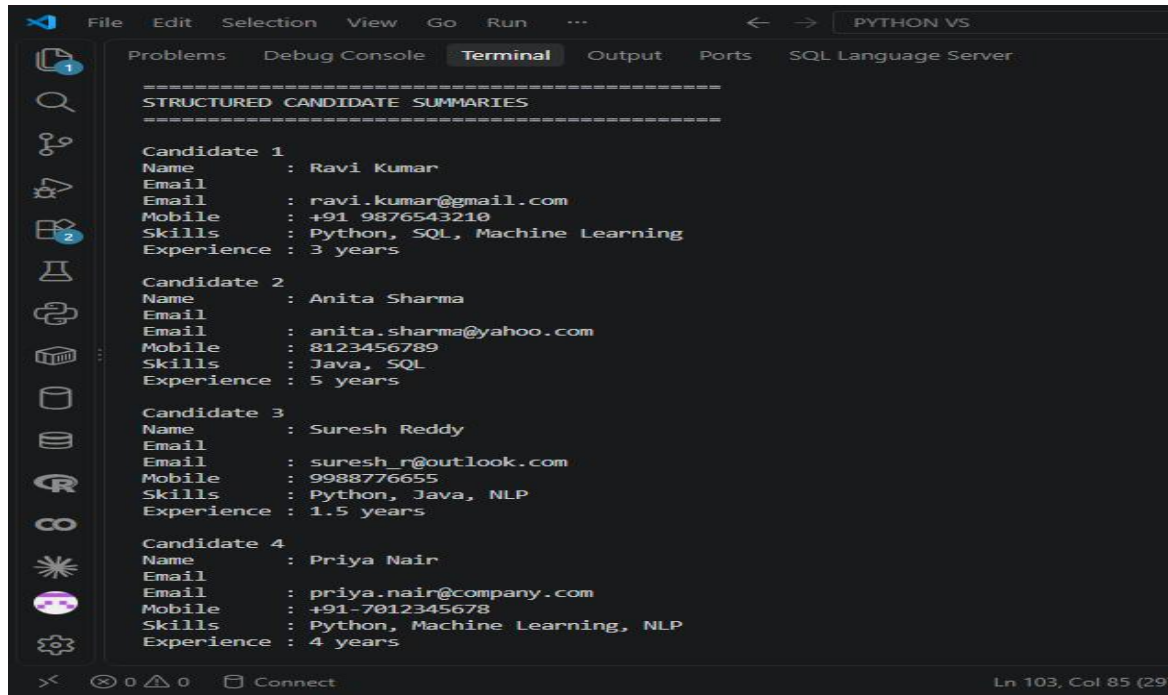
# ----- Run the system -----
candidates = [parse_resume(r) for r in resumes]

print("=" * 45)
print("STRUCTURED CANDIDATE SUMMARIES")
print("=" * 45)
for i, c in enumerate(candidates, 1):
    print(f"\nCandidate {i}")
    print(summary(c))

print("\n" + "=" * 45)
print("SHORTLISTED (Experience >= 2 yrs AND Python)")
print("=" * 45)
for c in candidates:
    if is_eligible(c):
        print(f"- {c['name']} | {c['experience']:g} yrs | {' '.join(c['emails'])}")

```

OUTPUT:



```
File Edit Selection View Go Run ... PYTHON VS
Problems Debug Console Terminal Output Ports SQL Language Server

=====
STRUCTURED CANDIDATE SUMMARIES
=====

Candidate 1
Name      : Ravi Kumar
Email     : ravi.kumar@gmail.com
Mobile    : +91 9876543210
Skills    : Python, SQL, Machine Learning
Experience : 3 years

Candidate 2
Name      : Anita Sharma
Email     : anita.sharma@yahoo.com
Mobile    : 8123456789
Skills    : Java, SQL
Experience : 5 years

Candidate 3
Name      : Suresh Reddy
Email     : suresh_r@outlook.com
Mobile    : 9988776655
Skills    : Python, Java, NLP
Experience : 1.5 years

Candidate 4
Name      : Priya Nair
Email     : priya.nair@company.com
Mobile    : +91-7012345678
Skills    : Python, Machine Learning, NLP
Experience : 4 years

Ln 103, Col 85 (297)
```

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

CODE:

```
import re

# ----- Product catalog -----
products = [
    "Samsung Galaxy Phone",
    "Apple iPhone 14",
    "Sony Wireless Headphones",
    "Dell Laptop",
    "HP Laptop Bag",
    "Phone Case",
    "Smartphone Stand",
    "Bluetooth Speaker",
    "Wireless Mouse",
```

```

    "Laptop Charger",
    "Headphone Stand",
    "Gaming Keyboard",
]

# ----- Pattern builders -----
BUILDERS = {
    "exact": lambda kw: rf"\b{re.escape(kw)}\b",
    "prefix": lambda kw: rf"\b{re.escape(kw)}\w*",
    "suffix": lambda kw: rf"\w*{re.escape(kw)}\b",
    "partial": lambda kw: re.escape(kw),
}

def search(items, mode, keyword, flags=0):
    pattern = BUILDERS[mode](keyword)
    regex = re.compile(pattern, flags)
    return pattern, [p for p in items if regex.search(p)]

# ----- Searches to perform -----
searches = [
    ("Exact match", "exact", "Phone", 0),
    ("Prefix search", "prefix", "Head", 0),
    ("Suffix search", "suffix", "less", 0),
    ("Partial search", "partial", "phone", 0),
    ("Case-insensitive", "partial", "PHONE", re.I),
]

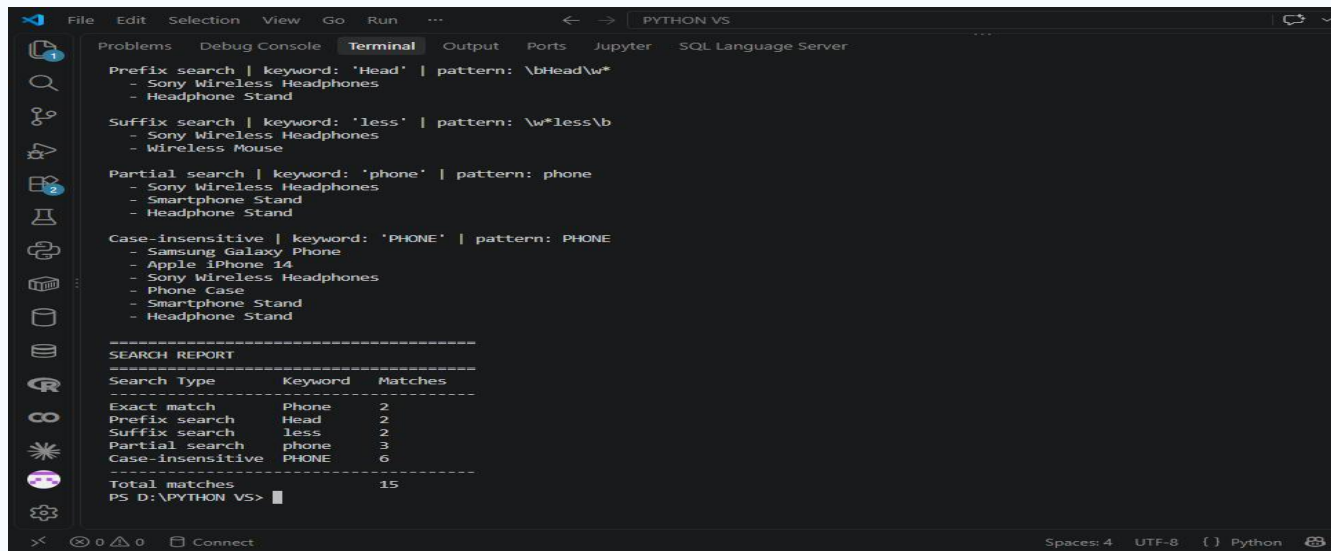
report = []

for label, mode, keyword, flags in searches:
    pattern, results = search(products, mode, keyword, flags)
    report.append((label, keyword, len(results)))

    print(f'{label} | keyword: '{keyword}' | pattern: {pattern}')
    for r in results:
        print(f' - {r}')
    if not results:
        print(" (no match)")
    print()

# ----- Report -----
print("=" * 38)
print("SEARCH REPORT")
print("=" * 38)
print(f'{"Search Type":<18} {"Keyword":<10} {"Matches":<10}')
print("-" * 38)
for label, keyword, count in report:
    print(f'{"label":<18} {"keyword":<10} {"count":<10}')
print("-" * 38)
print(f'{"Total matches":<28} {sum(c for _, _, c in report)}')
```

OUTPUT:



```
File Edit Selection View Go Run ... PYTHON VS
Problems Debug Console Terminal Output Ports Jupyter SQL Language Server

Prefix search | keyword: 'Head' | pattern: \bHead\w*
- Sony Wireless Headphones
- Headphone Stand

Suffix search | keyword: 'less' | pattern: \w*less\b
- Sony Wireless Headphones
- Wireless Mouse

Partial search | keyword: 'phone' | pattern: phone
- Sony Wireless Headphones
- Smartphone Stand
- Headphone Stand

Case-insensitive | keyword: 'PHONE' | pattern: PHONE
- Samsung Galaxy Phone
- Apple iPhone 14
- Sony Wireless Headphones
- Phone Case
- Smartphone Stand
- Headphone Stand

=====
SEARCH REPORT
=====
Search Type      Keyword  Matches
-----
Exact match      Phone    2
Prefix search    Head     2
Suffix search    less     2
Partial search   phone    3
Case-insensitive PHONE    6
-----
Total matches                15
PS D:\PYTHON VS>
```

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages for each field.
- Generate a final registration status report indicating whether the registration is successful.

CODE:

```
import re

# ----- Validation rules: field -> (pattern, format hint) -----
RULES = {
    "Register Number": (re.compile(r"\d{2}[A-Z]{3}\d{4}$"), "e.g. 21CSE1234"),
    "Email": (re.compile(r"^[A-Za-z0-9._]+@university\.edu$"), "use name@university.edu"),
    "Course Code": (re.compile(r"^[A-Z]{2,3}\d{3}$"), "e.g. CS101"),
    "Semester": (re.compile(r"^(?:Semester\s)?[1-8]"), "1 to 8"),
    "Mobile": (re.compile(r"^(?:\+91[\-s])?[6-9]\d{9}$"), "10 digits starting 6-9"),
}

# ----- Validation -----
def validate_student(student):
    """Return {field: (is_valid, message)} for one student."""
    results = {}
    for field, (pattern, hint) in RULES.items():
        value = student[field]
        if pattern.fullmatch(value):
```

```

        results[field] = (True, "Valid")
    else:
        results[field] = (False, f"Invalid ({hint})")
    return results

def registration_status(results):
    return "SUCCESS" if all(ok for ok, _ in results.values()) else "FAILED"

# ----- Sample students -----
students = [
    {"Register Number": "21CSE1234", "Email": "john.doe@university.edu",
     "Course Code": "CS101", "Semester": "3", "Mobile": "9876543210"},

    {"Register Number": "21cse12", "Email": "ravi@gmail.com",
     "Course Code": "CS101", "Semester": "5", "Mobile": "9123456780"},

    {"Register Number": "22ECE5678", "Email": "anita_s@university.edu",
     "Course Code": "EC3", "Semester": "Semester 9", "Mobile": "12345"},

    {"Register Number": "23MEC4321", "Email": "priya.n@university.edu",
     "Course Code": "ME204", "Semester": "Semester 2", "Mobile": "+91-7012345678"},
]

# ----- Run the system -----
report = []

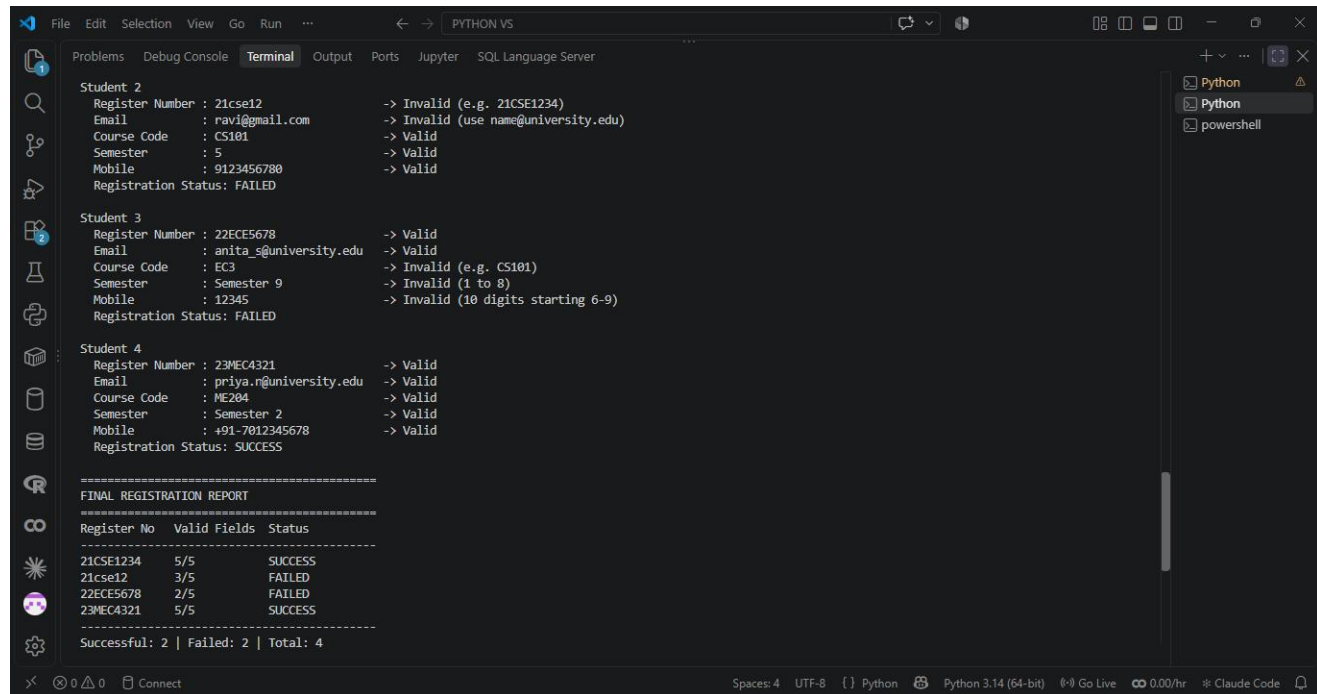
for i, s in enumerate(students, 1):
    results = validate_student(s)
    status = registration_status(results)
    valid_count = sum(ok for ok, _ in results.values())
    report.append((s["Register Number"], valid_count, status))

    print(f"Student {i}")
    for field, (ok, msg) in results.items():
        print(f" {field:<16}: {s[field]:<24} -> {msg}")
    print(f" Registration Status: {status}\n")

# ----- Final report -----
print("=" * 44)
print("FINAL REGISTRATION REPORT")
print("=" * 44)
print(f'{"Register No":<14} {"Valid Fields":<14} {"Status"}')
print("-" * 44)
for reg, count, status in report:
    print(f'{"reg":<14} {"str(count) + "/5":<14} {"status"}')
print("-" * 44)
success = sum(1 for _, _, st in report if st == "SUCCESS")
print(f'Successful: {success} | Failed: {len(report) - success} | Total: {len(report)}')

```

OUTPUT:



The screenshot shows a VS Code terminal window with the following output:

```
Student 2
Register Number : 21cse12      -> Invalid (e.g. 21CSE1234)
Email           : ravi@gmail.com -> Invalid (use name@university.edu)
Course Code    : CS101         -> Valid
Semester       : 5             -> Valid
Mobile         : 9123456780    -> Valid
Registration Status: FAILED

Student 3
Register Number : 22ECE5678    -> Valid
Email           : anita_s@university.edu -> Valid
Course Code    : EC3          -> Invalid (e.g. CS101)
Semester       : Semester 9    -> Invalid (1 to 8)
Mobile         : 12345         -> Invalid (10 digits starting 6-9)
Registration Status: FAILED

Student 4
Register Number : 23MEC4321    -> Valid
Email           : priya.n@university.edu -> Valid
Course Code    : ME204         -> Valid
Semester       : Semester 2    -> Valid
Mobile         : +91-7012345678 -> Valid
Registration Status: SUCCESS

=====
FINAL REGISTRATION REPORT
=====
Register No  Valid Fields  Status
-----
21CSE1234    5/5             SUCCESS
21cse12      3/5             FAILED
22ECE5678    2/5             FAILED
23MEC4321    5/5             SUCCESS
=====
Successful: 2 | Failed: 2 | Total: 4
```

The terminal window also shows a sidebar on the right with a file explorer containing 'Python' and 'powershell' files. The status bar at the bottom indicates 'Spaces: 4', 'UTF-8', 'Python', 'Python 3.14 (64-bit)', 'Go Live', '0.00/hr', and 'Claude Code'.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____